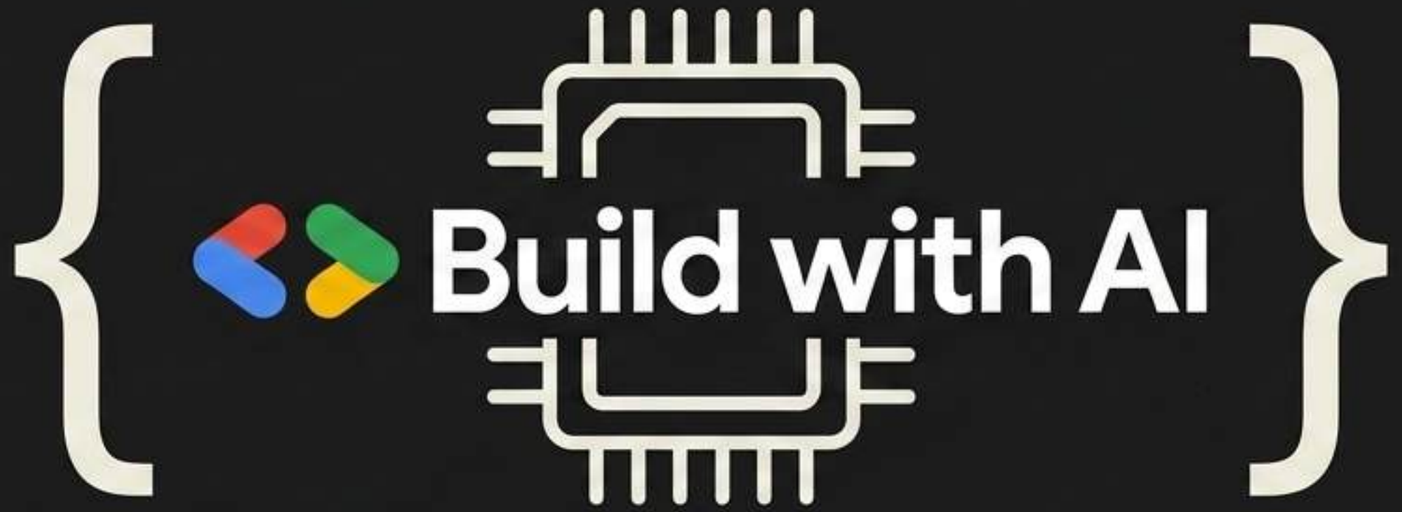


Whisper on the Edge

Local AI Transcription & Cloud Deployment

Build your own transcription service and keep your data private.

The Dilemma

Your file ⟶ Someone else's server

- ? storage duration
- ? model training
- ? who has access
- ? retention period

TurboScribe: *"We may retain your audio for up to 30 days"*
ElevenLabs: *"...to improve our Services"*

Cloud Services (11 Labs, Otter.ai)
Convenient, but your raw audio sits on external servers for months with uncertain privacy.

The Solution

Local-First Architecture

Key Beneficiaries:

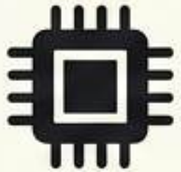
- ✅ **Corporations:** Protect IP & sensitive calls.
- ✅ **Government:** Maintain strict data control.
- ✅ **Healthcare & Legal:** Ensure confidentiality (HIPAA/privilege).

100% Data Privacy
Your audio never leaves your machine. No Internet, No API keys. Runs on modern CPUs (i5/Ryzen 5).

OpenAI Whisper

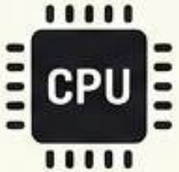
(High Accuracy, Heavy Compute)

The Engine: faster-whisper



INT8 Quantization

(Shrinks model size with minimal accuracy loss)



CPU Optimized

(Runs 2-4x faster directly on local CPUs—no expensive GPUs required)

The technology exists. We just needed a simple way to deploy it.

704 MB Support: Handles massive files securely.

No API Keys: Fully local processing.

Auto-Detect: Built-in Voice Activity Detection.

Per-Phrase Output: Generates instant .txt and .srt files.

edgescribe: Three Interfaces

1. Web UI

FastAPI + static HTML/JS

`make serve` → `http://localhost:8000`

Upload files, record voice, multi-file batch

2. CLI

Command line

```
python transcribe.py -i meeting.mp3
```

Batch processing, scriptable

3. REST API

For integration

`POST /v1/transcribe` → `job_id` → `poll for result`

Home Assistant, n8n, any script

Input & Output Formats

Input: MP3, WAV, FLAC, M4A, OGG, MP4, MKV, MOV...

Output: .txt + .srt (subtitles with timestamps)

The Developer's Blueprint

```python
# Lazy model loading with INT8 Quantization
model = WhisperModel(model_size, device="cpu", compute_type="int8")

# Streaming segment processing
segments, info = model.transcribe(audio_file)
for segment in segments:
    on_progress(segment.end / info.duration)
```

Lazy Loading
Cached in memory after the first cold start.

CPU / INT8
The magic flags enabling edge deployment.

on_progress
The callback that drives the real-time web UI progress bar.

CODE: HOW AUDIO GETS PROCESSED

```python
# core/engine.py — the transcription engine
_cache = {}

def get_model(model_name):
    if model_name not in _cache:
        from faster_whisper import WhisperModel
        _cache[model_name] = WhisperModel(
            model_name, device="cpu",
            compute_type="int8", cpu_threads=8
        )
    return _cache[model_name]

def transcribe(audio_path, model_name="large-v3-turbo",
                           language=None, on_progress=None):
    model = get_model(model_name)
    segments_iter, info = model.transcribe(
        audio_path, language=language,
        vad_filter=True,                    # ← Silero VAD
        vad_parameters={"min_silence_duration_ms": 500},
        beam_size=5,
    )
    segments = []
    for s in segments_iter:                 # ← streaming
        segments.append({
            "start": s.start, "end": s.end,
            "text": s.text.strip()
        })
        if on_progress:
            on_progress(s.end, len(segments))# ← live progress
    return segments, info.language
```

INT8 Quantization
Speeds up CPU inference 2-4x

Silero VAD
Filters out silence, improves accuracy

on_progress Callback
Feeds live progress to frontend

```python
# api.py — async job processing
from concurrent.futures import ThreadPoolExecutor
_executor = ThreadPoolExecutor(max_workers=2)

@app.post("/v1/transcribe")
async def submit(file: UploadFile, language="auto",
                 model="large-v3-turbo", speakers=0):
    tmp = save_to_temp(file)
    job_id = uuid4()
    JOBS[job_id] = {"status": "queued", ...}
    _executor.submit(_run, job_id, tmp, ...) # background
    return {"job_id": job_id}

# Client polls GET /v1/jobs/{job_id}
# Response includes: status, progress %, speed, ETA
```

ThreadPoolExecutor
Handles background job processing

```javascript
// static/app.js — frontend polling with smooth progress
pollTimer = setInterval(async () => {
  const job = await fetch(/v1/jobs/${jobId}).then(r => r.json());
  if (job.status === "processing") {
    targetPct = job.progress;          // smooth interpolation
  } else if (job.status === "done") {
    showResult(job);
    playChime(); // synthesized C-E-G chord
  }
}, 1000);
```

Smooth Interpolation
UI updates progress bar smoothly

Performance

Test: AMD Ryzen 7 PRO, 14 GB RAM, CPU only

Audio length	Time	Speed
10 min	~5 min	2x realtime
30 min	~15 min	2x
1 hour	~30 min	2x

Model Comparison

Model	Size	Speed	Quality
large-v3-turbo	1.5 GB	2x RT	Excellent ← default
large-v3	3 GB	1x RT	Best
medium	1.5 GB	4x RT	Good
small	500 MB	7x RT	OK
base	150 MB	15x RT	Basic

RAM usage: ~6 GB (model) + headroom
CPU load: 85-95% all cores

Docker Configuration

Dockerfile

```
# Dockerfile
FROM python:3.11-slim
RUN apt-get update && apt-get install -y ffmpeg
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "api.py", "--host", "0.0.0.0"]
```

docker-compose.yml

```
# docker-compose.yml
services:
  edgescribe:
    build: .
    ports: ["8000:8000"]
    volumes:
      - model-cache:/root/.cache/huggingface
    restart: unless-stopped
volumes:
  model-cache:
```

Nginx & Setup

/etc/nginx/sites-available/edgescribe

```
# /etc/nginx/sites-available/edgescribe
server {
    listen 443 ssl;
    server_name transcribe.yourdomain.com;

    ssl_certificate     /etc/letsencrypt/live/.../fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/.../privkey.pem;

    client_max_body_size 500M;  # large audio files

    location / {
        proxy_pass http://localhost:8000;
        proxy_read_timeout 3600;  # transcription takes minutes
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Let's Encrypt SSL

Essential for accepting massive uncompressed audio files without HTTP 413 errors.

Standard API timeouts will fail; AI transcriptions take minutes, not seconds.

Setup & API Key Protection

```
# Setup
apt update && apt install -y docker.io docker-compose nginx certbot
docker compose up -d
certbot --nginx -d transcribe.yourdomain.com

# API key protection
python api.py --api-key YOUR_SECRET
# or: export EDGESCRIBE_API_KEY=YOUR_SECRET
```

Scaling Matrix – Edge to Enterprise

	Minimal (1-10 Users)	Medium (10-100 Users)	Enterprise (1000+ Users)
Infrastructure	AWS EC2 / Hetzner CPX41 (Docker Compose)	Kubernetes (AWS EKS) or High-End Docker	Full Kubernetes Cluster
RAM	16 GB	64-128 GB	Custom / Multi-Node
Storage	50 GB+ NVMe	500 GB+ NVMe	Custom / Distributed
GPU	Optional / CPU-only	Recommended (e.g., T4/L4)	Essential (A100/H100 clusters)
Cost	~$120/mo (AWS) or €35/mo (Hetzner)	$200-$400/mo	Custom Setup
Throughput	2-3 concurrent users, queue up to 10 jobs	10-20 concurrent jobs, 20-40 hours audio/day	50-100 concurrent jobs, 1000 hours audio/day
Max File Size	2 hours	5 hours	10 hours (Requires audio chunking)

MLOps & Production Hardening

[x] OOM (Out of Memory) Prevention

Implement strict worker limits to prevent model loading spikes from crashing the server.

[x] Model Caching

Pre-load and hold whisper models in memory to eliminate ‘cold start’ latency for users.

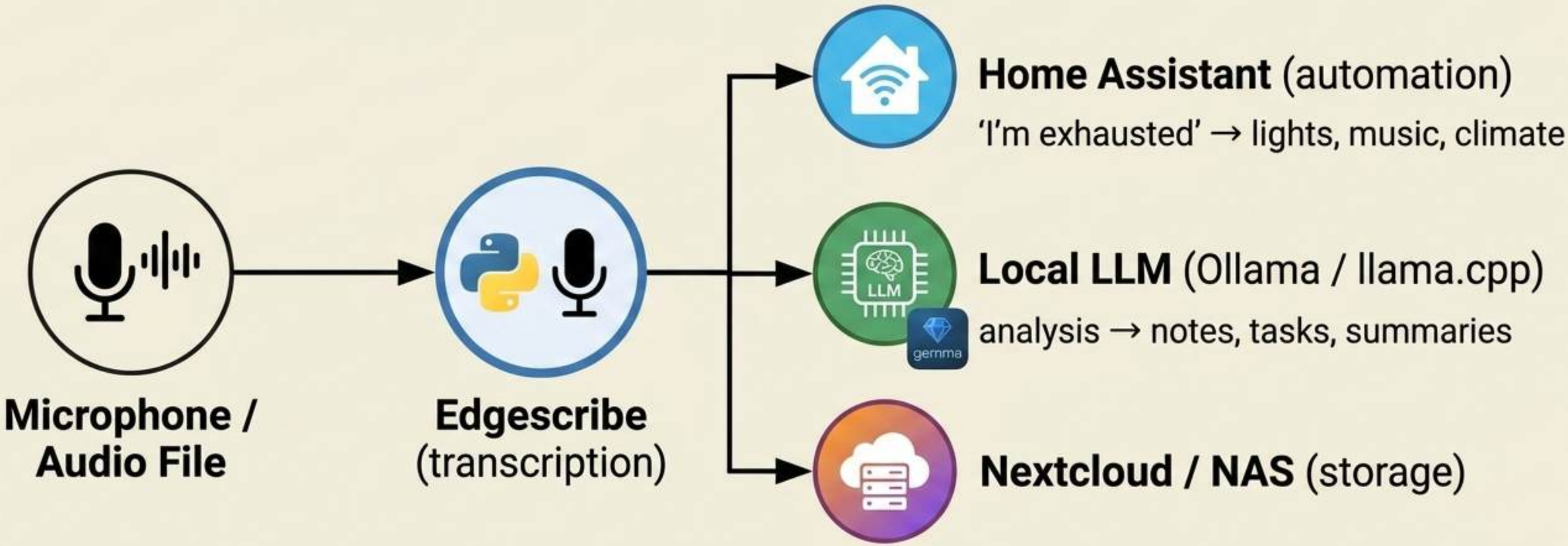
[x] Rate Limiting

Protect backend APIs from abusive batch upload scripts overloading the thread pool.

[x] SSL/TLS Security

Mandatory Let’s Encrypt reverse-proxy setup ensuring secure payload transit.

The Vision: The Local AI Stack



Internet: not needed at any step.

Transcription is just the first layer of a fully private AI stack.

Build the Local AI Movement

2026

> github.com/mikebionic/edgescribe

GDG

#WhisperAI #LocalFirst #Cloud #BuildWithAI
#GDG #GDGAshgabat